# C08 Prefabs

## CRE132 worksheet

### What you'll make

Reusable objects. A **prefab** is a GameObject saved as an asset: set it up once, put as many copies in your scenes as you like, and change them all at once by changing the prefab.

You'll make and edit a prefab, override one copy, make a **variant** (a prefab based on another one) and look at **nested** prefabs (prefabs inside a prefab). Then you'll write a **SlimeSpawner** that makes new slimes from a prefab while the game runs, with `Instantiate`, and removes them again with `Destroy`.



Figure 1: The finished scene: the player among two groves of trees, a slime, a bigger green slime, and slimes made by the spawner.

### Before you start

1. Unzip `CRE132_C08_Prefabs_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C08_Prefabs_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C08_Prefabs`.
3. Press **Play**. The C07 player walks and animates (WASD or arrows), and the slimes bob. Pressing Space does nothing yet. Press Play again to stop.

In the Hierarchy, the **Tree**, the two **Groves**, the **Slime** and **Slime Big** have **blue** names: they are prefab instances. Their prefab assets are in `Assets/_game/Prefabs/Characters` and `Assets/_game/Prefabs/Environment`. **Slime Spawner** is an empty object waiting for a script.

**Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

## 1. Making a prefab

In `Assets/_game/Art/Sprites/MysticWoods`, open `objects.png` and drag the `rock_1` sprite into the Scene view. Rename it **Rock**.

**Add it:** drag **Rock** from the Hierarchy into the `Assets/_game/Prefabs/Environment` folder. Its name turns blue: the object in the scene is now an *instance* of the **Rock** prefab asset.

Drag the **Rock** asset from the Project window into the scene twice more, so you have three rocks.

## 2. Editing the prefab

Double-click the **Rock** asset in the Project window. The scene is replaced by **Prefab Mode**: you're editing the asset itself. Change the Sprite Renderer's **Color** to a light blue. Then click the **<** arrow at the top left of the Hierarchy to go back to the scene.

**Test it:** all three rocks are blue. Change the asset, and every copy changes.

## 3. Overrides

Select just one rock in the scene and change its **Color** to red. That value now shows in **bold** with a blue bar beside it: it's an **override**, a change made only to this instance.

At the top of the Inspector, open the **Overrides** drop-down. **Revert All** throws your change away; **Apply All** copies it into the prefab, so every rock turns red. Choose **Revert All**.

## 4. A prefab variant

**Slime Big** is a **variant** of **Slime**. Select the **Slime Big** asset in `Prefabs/Characters`: its **Scale** is 1.6 and its **Color** is a yellow tint (R 1, G 1, B 0.45), which makes the blue slime look green. It stores only those differences; everything else, including the Animator, comes from Slime.

**Add it:** make your own. Right-click your **Rock** asset and choose **Create > Prefab Variant**. Name it **Rock Big**, open it and set its **Scale** to 2, 2, 1. Go back and drag a **Rock Big** into the scene.

**Test it:** open the base **Rock** prefab and change its Color again. Rock Big changes colour too, but keeps its own scale.

## 5. Nested prefabs

Double-click the **Grove** asset in `Prefabs/Environment`. It holds three **Tree** prefab instances and a **Bush**: prefabs inside a prefab. Go back to the scene.

**Test it:** open the **Tree** prefab and change its Sprite Renderer's **Sprite** to `tree_berries`. Go back: the lone Tree and every tree in both Groves now have berries. Open Tree again and set the Sprite back to `tree`.

## 6. Making copies at runtime: SlimeSpawner

Make a folder `Assets/_game/Scripts/C08_Prefabs` and create `SlimeSpawner` in it:

```
// Press Space to make a new slime from a prefab near this spawner.
// Each new slime is destroyed again after a few seconds, so the scene doesn't fill
// up.
using UnityEngine;
```

```
public class SlimeSpawner : MonoBehaviour
{
    // Drag the Slime prefab (or the Big Slime variant) from the Project window into
    // this slot.
    [SerializeField] private GameObject slimePrefab;
    [SerializeField] private float spawnRadius = 3f;
    [SerializeField] private float slimeLifetime = 4f;

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            SpawnSlime();
        }
    }

    public void SpawnSlime()
    {
        // A random point inside a circle around the spawner.
        Vector2 offset = Random.insideUnitCircle * spawnRadius;
        Vector3 spawnPosition = new Vector3(transform.position.x + offset.x,
            transform.position.y + offset.y, 0f);

        // Instantiate makes a copy of the prefab in the scene and gives it back to
        // us.
        GameObject newSlime = Instantiate(slimePrefab, spawnPosition,
            Quaternion.identity);

        // Destroy with a delay: the slime is removed slimeLifetime seconds from now.
        Destroy(newSlime, slimeLifetime);
    }
}
```

What's going on:

- **`GameObject slimePrefab`** is a slot for the thing to copy. You fill it in the Inspector.
- **`Random.insideUnitCircle`** gives a random point inside a circle of radius 1. Multiplying by `spawnRadius` makes the circle 3 units across each way.
- **`Instantiate(prefab, position, rotation)`** makes a copy of the prefab in the scene and returns it. `Quaternion.identity` means "no rotation".
- **`Destroy(newSlime, slimeLifetime)`** removes that copy after 4 seconds. Without the delay it would vanish straight away.

**Add it:** select **Slime Spawner** and **Add Component > SlimeSpawner**. Drag the **Slime** prefab *asset* from `Prefabs/Characters` in the **Project** window into the **Slime Prefab** slot. Leave **Spawn Radius** at **3** and **Slime Lifetime** at **4**.

**Test it:** press Play and press **Space** a few times. Slimes appear around the spawner and bob, and **Slime(Clone)** objects appear in the Hierarchy. Four seconds later each one disappears.

## Check your work

- ☐ Your **Rock** prefab is in `Prefabs/Environment`, and changing it changes every rock.
- ☐ An override on one rock shows in bold and can be reverted.
- ☐ **Rock Big** is bigger but follows Rock's colour.
- ☐ Changing the Tree prefab changes the trees inside both Groves.
- ☐ Space makes a slime near the spawner that disappears after 4 seconds.
- ☐ Compare with `CRE132_C08_Prefabs_Finished` if something behaves differently.

## Key points

- **A prefab is a saved GameObject you can reuse.** Change the asset and every copy changes; change one copy and you get an override.
- **Variants** inherit from a base prefab and store only what's different.
- **`Instantiate`** copies a prefab (or any GameObject) into the scene. **`Destroy`** removes a GameObject (or a component), now or after a delay.
- **Drag prefabs from the Project window** into script slots. An object from the scene would be gone when the scene unloads.

## If something goes wrong

- **UnassignedReferenceException: "The variable slimePrefab of SlimeSpawner has not been assigned":** the Slime Prefab slot is empty.
- **Space does nothing:** SlimeSpawner isn't on **Slime Spawner**, or the Game view doesn't have keyboard focus: click inside it once after pressing Play.
- **My change only affected one object:** you changed an instance (an override). Use **Overrides > Apply All**, or edit the prefab asset in Prefab Mode.
- **The name isn't blue:** the object isn't connected to a prefab. Drag it into a Prefabs folder to make one.

## Try it

1. Drag **Slime Big** into the spawner's slot instead.
2. Spawn a slime every 2 seconds automatically, with a timer in `Update` (add `Time.deltaTime` to a float and reset it when it passes 2).
3. Make the spawner give each new slime a random colour: `newSlime.GetComponent<SpriteRenderer>().color = ...`